

임장혁

실시간·비동기 시스템의 상태와 데이터 정합성을 설계하는 백엔드 개발자입니다.

Java/Spring과 TypeScript/NestJS를 사용해 메시지 전파, 런타임 상태, LLM 흐름처럼 실패 조건이 사용자 경험으로 이어지는 문제를 다뤄왔습니다. 문제를 재현하고 선택한 구조의 효과를 테스트와 수치로 확인합니다.

백엔드 개발자 Java · Spring Boot · TypeScript · NestJS · LLM github.com/imjanghyeok dlaljh.tistory.com
qkdrhkgn23@naver.com

문제를 바라보는 방식

기능이 동작하는지보다, 실패했을 때 사용자가 무엇을 잃는지를 먼저 봅니다.

- 사용자에게는 지연, 반복, 누락이 모두 신뢰 하락으로 보일 수 있다고 가정합니다.
- 런타임 상태와 영속 데이터를 분리하고, 재시작·재접속·시간 초과 이후 복구 가능성을 확인합니다.
- 선택한 구조가 감수한 비용까지 기록해 다음 운영 판단의 기준으로 남깁니다.

주요 경험 축

- **상태 모델링** Redis 접속 상태, TTL 세션, 스케줄러 상태 전이
- **흐름 제어** Kafka 순서 보장, WebSocket 다중 전달, LLM 종료 정책
- **책임 분리** 도메인 객체, 소비자, 서버 정책의 변경 이유 분리
- **검증** 테스트 코드, 배포 설정, 로컬 실행 기록 기준 검증

프로젝트에서 반복적으로 다룬 기술 영역

백엔드 Java · Spring Boot · NestJS · Node.js · TypeScript

메시지 / 캐시 Kafka · Redis · WebSocket · STOMP

데이터 MySQL · PostgreSQL · MongoDB · JPA · TypeORM

인프라 Docker · Docker Compose · GitHub Actions · AWS · NCP · Nginx

01. Synapse

AI 면접 시뮬레이션 서비스

2025.01.05 ~ 2026.02.06 · 백엔드 · NestJS · TypeORM · MySQL · Docker · GitHub Actions · NCP VPC

이 프로젝트를 한 줄로 설명하면

LLM이 만든 질문을 그대로 믿지 않고, 서버가 면접 흐름을 통제하도록 만든 프로젝트

1. 프로젝트 개요

사용자의 포트폴리오를 기반으로 AI 면접관이 개인화된 질문과 피드백을 제공하는 서비스입니다.

내 역할: 면접 흐름 제어 API, 질문 생성 상태 관리, TTL 정리, 문서 삭제 정책, CI/E2E, Docker 배포와 NCP VPC 구성을 담당했습니다.

네트워크 구성: Public Subnet의 Bastion/Application 서버와 Private Subnet의 MySQL을 분리하고, ACG와 사설 IP 기반 접근 규칙으로 운영자 SSH와 애플리케이션의 DB 접근 경로를 제한했습니다.

[GitHub 저장소](#) · [CI 품질 게이트 기록](#) · [TTL 스케줄링 기록](#)

도전 과제

프롬프트가 좋아도 LLM 출력은 매번 달라질 수 있습니다. 짧거나 애매한 답변에서는 비슷한 질문이 반복되거나 면접이 어색하게 끝났고, 이를 사용자에게 보이지 않는 내부 변동으로만 둘 수 없었습니다.

해결 방향

질문 후보 생성은 LLM에 맡기되, 종료·중복·재시도 여부는 서버 정책이 판단하도록 역할을 나눴습니다.

400회

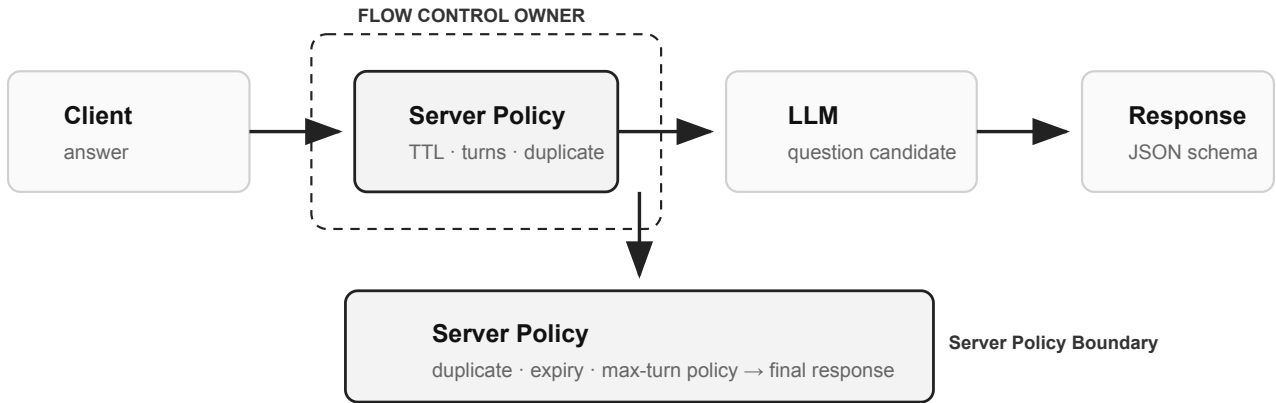
LLM 질문 생성 시도

396회 성공 · Temperature 0.4~0.7

1.01%

완전 일치 중복률

Temp 0.7 · 유형 분류 15.1%는 별도 지표



LLM은 질문 후보를 생성하고, 서버 정책이 중복·종료·재시도 흐름을 최종 결정합니다.

상세 설계

제어 계층

문제 1. LLM 비결정성으로 면접 흐름이 깨지는 문제

사용자에게 반복 질문과 어색한 종료는 “AI가 나를 이해하지 못한다”는 신호입니다. LLM 출력은 후보로만 두고, 중복/종료 판단은 서버 정책으로 제한했습니다.

실제로 부딪힌 문제
테스트 중 짧은 답변이나 애매한 답변이 들어오면 꼬리질문 품질이 급격히 낮아지고, 종료 신호가 늦어져 비슷한 질문이 반복될 수 있었습니다. 그래서 LLM의 `isLast`를 그대로 믿지 않고 서버가 최종 종료 조건을 다시 계산했습니다.

기존 상태

프롬프트 조정만으로는 중복, 종료, 재시도 실패를 안정적으로 통제하기 어려웠습니다.

바꾼 방식

LLM은 질문 후보를 만들고, 서버는 종료·중복·재시도 정책을 판단하도록 분리했습니다.

핵심 구현

서버 제어 덮어쓰기 정책

종료 모드

JSON Schema

실험 결과: 400회 시도 중 396회가 성공했습니다. 성공 응답의 원문 완전 일치 중복률은 Temperature 0.4의 2.04%에서 0.7의 1.01%로 집계됐고, 별도 질문 유형 분류 기준으로는 57.1%에서 15.1%였습니다. 후자는 의미 중복을 직접 판정한 값이 아니라 유형 분류 기반 지표로 분리해 기록했습니다.

세션과 정리

문제 2. 면접 중 임시 상태를 어디에 저장할 것인가

면접 중 상태가 남으면 메모리 누수와 잘못된 중복 판단으로 이어집니다. 그래서 면접 진행 상태를 TTL이 있는 런타임 상태로 정의했습니다.

예상과 달랐던 점

처음에는 세션별 타이머가 가장 단순해 보였지만, 면접 세션이 늘면 타이머 누수와 정리 시점을 추적하기 어려워질 수 있었습니다. 만료 시각 기준으로 한 곳에서 정리하는 구조가 필요했습니다.

기존 상태

질문 이력, 방문 주제, 톤 수는 면접이 끝나면 즉시 사라져야 하는 상태였습니다.

바꾼 방식

상태 보관은 `KeySetStore`, 만료 관리하는 `MinHeapScheduler`가 소유하게 했습니다.

핵심 구현

KeySetStore

MinHeapScheduler

TTL 정리

달라진 점: 영속 데이터와 흐름 제어용 임시 상태의 저장 계층을 분리했습니다.

문제 3. 연관 엔티티 삭제 시 정합성과 성능을 동시에 보장하는 방법

연관 삭제 정합성은 DB 제약으로 보장하고, 서비스 계층은 삭제 요청 검증과 응답 책임에 집중시켰습니다.

기존 상태

문서 삭제 시 자식 데이터 누락과 대량 삭제 비용 증가가 함께 발생할 수 있었습니다.

바꾼 방식

삭제 정합성은 DB에 위임하고, 애플리케이션은 정책 검증에 집중했습니다.

핵심 구현

onDelete: CASCADE

delete()

문서 도메인 관계

달라진 점: 연관 데이터 삭제 책임을 DB 제약으로 옮기고, 문서 일괄 삭제 API와 E2E 테스트를 추가했습니다.

운영과 한계

운영 보강

마이그레이션: 운영 환경에서 `synchronize` 의존을 제거하고, 스키마 변경을 TypeORM 마이그레이션 파일과 배포 절차 안에서 검토하도록 전환했습니다.

CI/E2E: AI API가 포함된 인터뷰 흐름을 수동 확인에만 맡기지 않기 위해 Lint, Build, Unit, E2E 단계를 분리한 품질 게이트를 구성했습니다.

Docker: 1GB RAM 서버 환경에서 배포 안정성을 고려해 `pnpm deploy`, Dockerfile 분리, 배포 워크플로를 정리했습니다.

NCP VPC: 애플리케이션 서버를 Public Subnet의 Bastion 역할로 두고 MySQL을 Private Subnet에 배치했습니다. Bastion에서 Private DB로 연결되지 않는 문제는 ACG, 라우팅, SSH 서비스, MySQL `bind-address`, 계정 host 권한과 `DB\_HOST` 설정을 순서대로 확인했습니다.

팀 협업 방식: 설계 이슈와 코드 리뷰에 실패 조건을 같이 남겨 “왜 이 구조를 택했는지”를 팀원이 따라갈 수 있게 했습니다.

운영 판단

단일 인스턴스 검증 단계에서는 인메모리 TTL로 충분했습니다. 운영 전환 기준은 동시 면접 세션 증가, 인스턴스 2대 이상으로 확장되는 시점, 세션 유실을 허용할 수 없는 시점입니다. 이때 Redis 세션 저장소로 이전하고, LLM 시간 초과·재시도 횟수·대체 종료·중복 방지 실패 로그를 모니터링 대상으로 뒀야 합니다.

실제 구현

인메모리 TTL 세션 저장소, MinHeapScheduler, 서버 제어 종료 정책, TypeORM Cascade 삭제, CI/E2E, Docker 경량화

이후 고려한 방향

Redis 세션 저장소, LLM 시간 초과/대체 응답, 출력 평가기, 중복 방지 장치, 검색 기반 맥락 주입

회고

확률적 출력을 서비스에 넣을 때 핵심은 더 긴 프롬프트가 아니라, 실패했을 때 서버가 어디서 멈추고 복구할지 정하는 일이었습니다.

02. SmileTogether

MSA 기반 협업 메신저

2025.01 ~ 2025.03 · 백엔드 · Spring Boot · Kafka · MongoDB · Redis · FCM · Docker Compose

이 프로젝트를 한 줄로 설명하면

분산 채팅에서 메시지 전달과 조회 시점의 차이를 줄인 프로젝트

1. 프로젝트 개요

Slack형 협업 메신저를 MSA 구조로 구현한 프로젝트입니다. 핵심 개발 인원은 4명이며, 팀장으로서 백엔드 주담당을 맡아 채팅, 히스토리, 멤버, 알림 서버의 연동을 담당했습니다.

내 역할: WebSocket/STOMP 채팅 흐름, Kafka 메시지 전파, MongoDB 기반 히스토리 조회, KafkaLagChecker, FCM 알림 서버, 서버 간 JWT 전달 흐름을 구현했습니다.

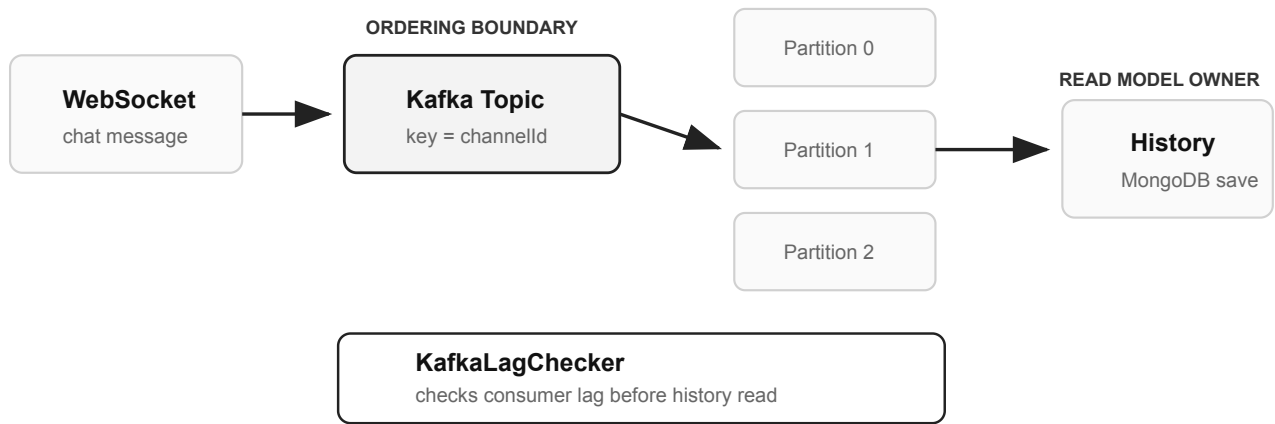
도전 과제

서버가 여러 대로 늘어나면 WebSocket 연결 위치와 메시지 저장 시점이 달라집니다. 메시지는 다른 서버의 사용자에게도 전달되어야 하고, 전송 직후 조회할 때는 아직 저장되지 않은 메시지가 누락처럼 보이지 않아야 했습니다.

해결 방향

서버 간 전파는 Kafka로 분리하고, Channel ID를 key로 사용해 동일 채널 이벤트를 같은 partition으로 라우팅했습니다. History 조회 전에는 consumer lag을 최대 500ms 확인해 비동기 저장 지연으로 인한 직후 조회 누락 가능성을 낮췄습니다.

히스토리 API 조회 경로 보강 지연 확인 후 DB 조회	400 → 80 외부 프로필 호출 요청 내 발신자별 캐시 · 80% 감소	29.34% History p95 개선 92.46ms → 65.33ms · 예비 측정	5 × 100ms 지연 확인 최대 5회 오프셋 지연 확인
Channel ID partition 라우팅 기준 partition 내 레코드 순서			



사용자가 체감하는 순서 범위를 채널 단위로 정의하고, Channel ID를 Kafka key로 사용해 동일 채널 이벤트를 같은 partition으로 라우팅했습니다.

상세 설계

메시지 전파

핵심 흐름. 다중 서버에서 메시지가 같은 경험으로 도착해야 했다

다른 서버에 연결된 사용자가 메시지를 받지 못하면 사용자에게는 장애로 보입니다. 서버 간 직접 호출 대신 Kafka를 이벤트 경계로 두었습니다.

실제로 부딪힌 문제

WebSocket 연결이 특정 서버에 붙어 있기 때문에, 서버가 늘어나는 순간 “내 서버의 구독자”와 “다른 서버의 구독자”가 갈라졌습니다. 직접 호출 방식은 서버 수가 늘수록 전파 경로와 실패 지점이 함께 늘어났습니다.

기존 상태

WebSocket 연결은 특정 인스턴스에 붙어 있어 서버 확장 시 전파 경로가 끊길 수 있었습니다.

바꾼 방식

Kafka 토픽을 서버 간 이벤트 경계로 두고 메시지 다중 전달 책임을 분리했습니다.

핵심 구현

STOMP 처리기

Kafka 생산자/소비자

달라진 점: 채팅 서버 확장 이후에도 메시지 전파 책임이 동일한 구조로 유지됐습니다.

순서와 동기화

상세 설계 1. 순서 보장 범위를 채널 단위로 제한

사용자가 체감하는 순서 보장 범위를 “채널 내부”로 제한하고, Kafka key를 Channel ID로 결정했습니다.

기존 상태

순서를 보장해야 하는 단위를 먼저 정하지 않으면 Kafka 파티션 전략이 흔들렸습니다.

바꾼 방식

“같은 채널 안의 대화”를 순서 보장 범위로 정의했습니다.

핵심 구현

channelId 키

파티션 partition 내 레코드 순서

달라진 점: Kafka 순서 보장을 기술 기능이 아니라 사용자 대화 맥락 기준으로 설계했습니다.

상세 설계 2. 저장 지연이 누락처럼 보이는 구간 줄이기

전송 직후 히스토리에 메시지가 없으면 사용자는 “보낸 메시지가 사라졌다”고 느낍니다. DB 조회 전에 consumer lag을 확인해 저장 지연과 저장 실패를 관측 관점에서 구분할 단서를 마련했습니다.

디버깅 흐름

메시지 전송은 성공했는데 직후 히스토리 조회에서 누락되는 사례를 확인했습니다. DB 저장 실패가 아니라 소비자가 아직 처리하지 못한 전파 지연이었고, 최신 오프셋과 커밋된 오프셋의 차이를 확인하는 방식으로 원인을 분리했습니다.

기존 상태

실제 유실이 아니어도 저장 지연은 사용자에게 메시지 유실처럼 보였습니다.

바꾼 방식

History API가 조회 전에 consumer lag을 최대 500ms 확인해 비동기 저장 지연으로 인한 직후 조회 누락 가능성을 낮췄습니다.

핵심 구현

KafkaLagChecker

최신 오프셋

커밋된 오프셋

달라진 점: 비동기 저장 구조를 유지하면서 직후 조회에서 저장 지연이 누락처럼 보일 가능성을 낮췄습니다. 특정 메시지의 저장 완료를 보장하는 기능으로 확대 해석하지 않았습니다.

상세 설계 3. History 프로필 중복 호출을 요청 범위에서 제거

메시지 20개를 조회할 때 같은 발신자의 프로필을 메시지마다 다시 요청하고 있었습니다. 요청 하나의 범위에서 `senderId`별 결과를 재사용해 외부 호출 수를 줄였습니다.

측정 조건

동일 Docker 환경·데이터·Smoke 프로필에서 Before/After를 비교했습니다. Space Server가 기동되지 않은 fallback 경로이며, 각 조건 1회 실행인 예비 측정입니다.

기존 상태

20개 메시지 조회 시 발신자가 반복되어도 외부 프로필 호출을 매번 수행했습니다.

바꾼 방식

지역 `Map`에 성공 결과와 `null` 결과를 함께 저장해 같은 요청 안의 중복·실패 재호출을 막았습니다.

핵심 구현

senderId별 요청 범위 캐시

실패 결과 재사용

History 부하 테스트

예비 측정: HTTP 성공률은 Before/After 모두 100%였고, 외부 프로필 호출은 400회에서 80회로 80% 감소했습니다. History p95는 92.46ms에서 65.33ms로 29.34% 개선됐습니다. 실제 Space Server와 3회 이상 반복 측정 전까지 공식 기준선으로는 사용하지 않습니다.

상세 설계

책임 경계

상세 설계 4. 실시간 경로와 조회 모델 분리

채팅 서버는 실시간 전달에 집중하고, 저장/조회/삭제 정책은 History Server로 분리했습니다.

기존 상태

실시간 전송과 히스토리 관리는 데이터 모델과 변경 이유가 달랐습니다.

바꾼 방식

Kafka 이벤트를 경계로 서버 책임을 분리했습니다.

핵심 구현

히스토리 서버

MongoDB 문서

달라진 점: 채팅 서버 변경 없이 히스토리 조회 정책을 발전시킬 수 있는 구조가 됐습니다.

보조 구현

프로필/JWT: 채팅·히스토리 서버에서 고정 프로필 값을 제거하고, 서버 간 HTTP 호출과 HTTP/STOMP 헤더 양쪽의 JWT 추출 흐름을 정리했습니다.

알림: 채팅 서버가 알림 책임까지 가지 않도록 Kafka 이벤트를 소비하는 알림 서버 시범 구현과 FCM 웹 푸시 전송 흐름을 분리했습니다.

검증: k6·Node.js·Docker Compose 기반 Smoke/부하 테스트와 Chat/History 테스트를 통과했고, Prometheus에서 Chat·History target 수집을 확인했습니다. Notification Server는 Firebase 자격 증명과 실행 서버가 없어 실제 부하 검증에서 제외했습니다.

Docker Compose: chat-server 단독, history-server 단독, 통합 실행 환경을 분리해 MSA 로컬 실행 재현성을 높였습니다.

운영 판단

Channel ID 키는 채널 내 적합성을 얻는 대신 특정 파티션에 부하가 몰릴 위험을 감수합니다. 운영에서는 파티션별 지연, 특정 채널의 트래픽 편중, 조회 확인 시간 초과 비율을 봐야 합니다. 최대 500ms 이후에도 지연이 남으면 대체 응답을 주고, 재처리와 멍등 메시지 키로 중복 저장을 막는 기준이 필요합니다.

실제 구현

Kafka 기반 WebSocket 다중 전달, Channel ID 키 순서 보장, KafkaLagChecker 반복 확인, 히스토리 서버 분리

이후 고려한 방향

파티션 부하 집중 감시, 시간 초과 대체 응답, 소비자 장애 재처리, 멍등 메시지 처리

회고

분산 채팅에서 전송 성공, 저장 완료, 조회 가능 시점은 서로 다른 문제였습니다. 사용자 신뢰는 이 간극을 얼마나 짧고 예측 가능하게 만드는지에 달려 있었습니다.

03. FaceFriend

AI 이미지 분석 기반 데이팅 커뮤니티

2024.03 ~ 2024.06 · 백엔드 · Spring Boot · WebSocket · STOMP · Redis · JPA

이 프로젝트를 한 줄로 설명하면

접속 상태와 현재성을 기준으로 메시지 재전달과 실패 피드백을 검증한 프로젝트

1. 프로젝트 개요

Spring Boot 기반 AI 이미지 분석 데이팅 커뮤니티에서 WebSocket/STOMP 채팅 기능과 채팅 응답 구조 개선을 담당했습니다.

협업 규모: 6명(AI 2명, 프론트엔드 2명, 백엔드 2명) · **내 역할:** 채팅방/메시지 도메인, Redis 소켓·채팅 상태 저장소, 미수신 메시지 재전달, STOMP 예외 응답, 채팅방 상태별 DTO를 구현했습니다.

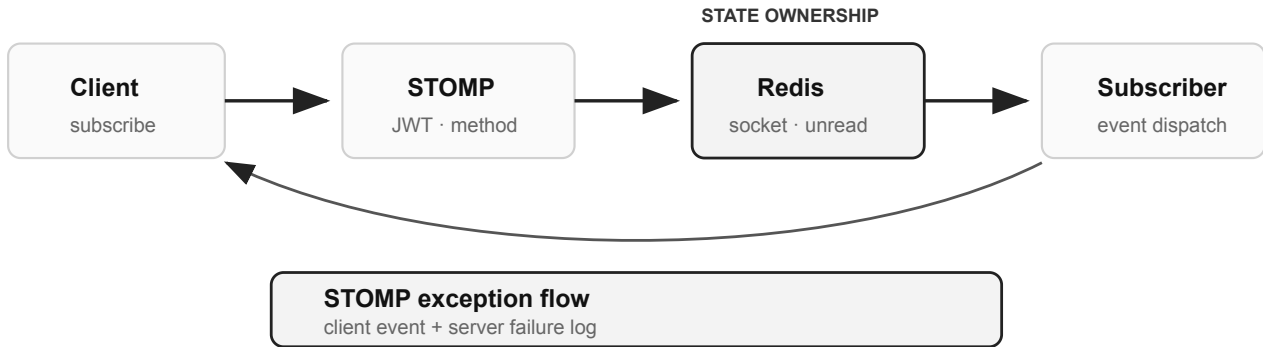
도전 과제

정상적인 disconnect/reconnect 이벤트가 도달하는 조건에서 앱의 접속 상태에 따라 전달 경로를 나눠야 했습니다. 비정상 종료나 네트워크 단절, heartbeat·ACK 전 장애까지의 전달 보장은 구현 범위에서 제외했습니다.

해결 방향

정상적인 접속 상태가 확인되면 온라인 사용자는 STOMP로 즉시 전달하고, 오프라인 사용자의 메시지는 Redis List에 보관한 뒤 재접속 시 connectChat 이벤트로 전달했습니다. 비정상 종료·heartbeat·ACK 기반 전달 보장은 후속 과제로 분리했습니다.

Presence	Redis List	STOMP 오류
소켓 접속과 채팅방 입장 상태의 현재성 분리	미접속 메시지 보관과 재접속 복구	백엔드 실패를 프론트엔드 이벤트로 전달



영속 메시지는 DB에, 접속/입장 상태는 Redis에 두어 실시간 판단과 저장 책임을 분리했습니다.

상세 설계

현재성과 오류

핵심 흐름. 접속 상태가 틀리면 메시지 전달 의미도 틀어진다

정상적인 **disconnect/reconnect** 이벤트가 도달하는 조건에서 온라인·오프라인 상태에 따라 즉시 전달과 보관·재전달 경로를 나누는 것이 핵심이었습니다.

실제로 부딪힌 문제

브라우저 종료나 네트워크 단절처럼 정상 연결 해제가 오지 않는 상황에서는 접속 상태가 남을 수 있습니다. 오래된 접속 상태가 남으면 상대가 방에 있는 것처럼 판단되어 메시지가 온라인 사용자에게 잘못 전달되거나 오프라인 보관 대상에서 빠질 수 있었습니다.

기존 상태

접속 상태를 오래 보관되는 사실처럼 취급하면 비정상 종료 뒤에도 사용자를 접속 중으로 판단할 수 있었습니다.

바꾼 방식

Redis를 현재 상태 저장소로 사용하고, 접속 중이면 STOMP 즉시 전달, 미접속이면 Redis List 보관 후 정상 재접속 시 재전달하도록 경로를 나눴습니다. 비정상 종료와 ACK 전 장애는 보장 범위에서 제외했습니다.

핵심 구현

SocketInfoRedisRepository

ChatRoomInfoRedisRepository

온라인·오프라인 전달 흐름

달라진 점: 정상적인 재접속 흐름에서 앱의 접속 상태를 기준으로 메시지 전달 경로를 분리하고, 오프라인 메시지를 재전달할 수 있게 했습니다.

상세 설계 1. 실패도 실시간 이벤트로 전달

메시지 전송 실패가 화면에 보이지 않으면 사용자는 입력이 무시됐다고 느낍니다. HTTP 예외 흐름에 머물러 있던 실패를 STOMP 이벤트로 변환해 프론트엔드가 실시간으로 처리하도록 했습니다.

구현 중 어려움

HTTP 요청과 달리 STOMP 메시지에는 일반적인 HTTP 응답 코드가 없으므로, 백엔드 예외를 프론트엔드가 받을 수 있는 메시지 이벤트로 변환해야 했습니다. 실패 응답의 목적지와 payload 형식도 정상 이벤트와 구분할 수 있어야 했습니다.

기존 상태

백엔드 예외가 HTTP 예외 처리기에만 도달해 STOMP 클라이언트에는 실패 사실이 전달되지 않았습니다.

바꾼 방식

백엔드가 오류 payload를 STOMP로 먼저 발행하고, 프론트엔드는 해당 이벤트를 구독해 사용자에게 즉시 실패 상태를 반영하도록 했습니다.

핵심 구현

STOMP 오류 응답

프론트 오류 이벤트

달라진 점: 백엔드의 실패를 프론트엔드가 실시간 이벤트로 인지할 수 있게 하면서, 성공·실패 응답의 처리 경계를 명확히 했습니다.

운영 판단

복구 기준

운영 판단

접속 상태는 빠른 조회보다 오래된 상태 제거가 더 중요합니다. 하트비트 누락, 연결 TTL 만료, 재접속 발생률을 기준으로 정리를 판단해야 합니다. Redis 장애나 서버 재시작 이후에도 DB 메시지를 기준으로 미수신 메시지를 재구성할 수 있는 대체 경로가 필요합니다.

실제 구현

Redis 소켓/채팅 상태 저장소, 정상 접속 상태 기반 전달, STOMP 오류 이벤트, 메서드 기반 이벤트 표준화

이후 고려한 방향

하트비트, 연결 TTL, 오래된 상태 정리, 재접속 복구, 미수신 메시지 재전달 정책

회고

접속 상태는 저장된 사실보다 현재성이 중요했습니다. 정상적인 접속 상태를 기준으로 즉시 전달과 재접속 재전달을 나누고, 실시간 통신의 실패도 프론트엔드가 처리할 수 있는 이벤트로 설계한 점이 핵심이었습니다.

04. 기타 프로젝트 및 활동

설계 학습, 해커톤, Django 프로젝트, Java 학습 기록

KiKi Kiosk · OneHabit · Moyeo · Relanz · Woowacourse Precourse

왜 중요했나

핵심 프로젝트 이전에도 요구사항을 사용 사례와 다이어그램으로 정리하고, 제한된 시간 안에서 기능 범위를 재정의하며, 실제 사용자의 접근성과 운영 환경까지 고려하는 경험을 반복했습니다.

OOP

GRASP

설계 문서

KiKi Kiosk · 설계 문서에서 객체 책임 도출

정보 취약계층을 위한 키오스크를 만들며 사용 사례, 순서도, 시스템 순서도, 클래스 다이어그램을 작성했습니다. 관리자/고객 기능을 분리하고 Controller, Creator, Expert, DTO 패턴을 적용해 주문·결제·정산 책임을 나눴습니다.

github.com/imjanghyeok/kiki_kiosk

해커톤

기획

일정 관리

OneHabit · 문제 정의와 인증 상태 관리

5 Whys로 서비스 문제를 정의하고 대표 사용자 흐름을 정리했습니다. 습관 CRUD, 활동 인증, 경험치/성장 로직, 하루 3회 인증 제한, 자정 인증 초기화와 연속 기록 초기화를 구현했습니다.

github.com/9oormthon-univ/2024_BEOTKKOTTHON_TEAM_19_BE

DJANGO

DRF

추천

Moyeo · 액티브 시니어 공동구매 플랫폼

모임 CRUD, 모임 등급 자동 승급, 클럽 태그 기반 상품 추천, 알림 내역 조회 API를 구현했습니다. Swagger로 API 명세를 자동화하고 Docker Compose 설정을 정리해 초기 데이터 주입과 개발 환경 구성을 보완했습니다.

github.com/CalmCrews/silver_backend

DJANGO

인증

배포

Relanz · 스트레스 관리 챌린지 플랫폼

회원가입/로그인, 챌린지 태그, 아바타 커스터마이징, 스코어·랭킹, 설문/활동 태그 기반 추천 기능을 구현했습니다. Nginx, Unicorn, AWS EC2로 배포와 도메인 연결을 경험했습니다.

github.com/CalmCrews/Relanz

JAVA

OOP

코드 리뷰

우아한테크코스 프리코스 · Java 설계 훈련

계산기, 자동차 경주, 로또 미션에서 요구사항 분석, 기능 목록, 테스트 목록, 시퀀스 다이어그램을 작성했습니다. MVC, Strategy, Validator 분리를 적용하고 코드 리뷰를 통해 책임 분리와 가독성을 논의했습니다.

github.com/imjanghyeok/java-racingcar-7